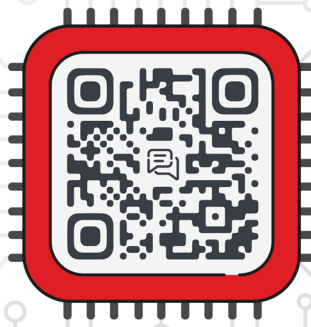




5



5



750 sp

نظري

كلية الهندسة المعلوماتية

السنة الأولى

المصفوفات

د. مدحت الصوص

د. فراس ضعيف



برمجة ا

RB Informatics; 21/12/2025

محتوى مجاني غير مخصص للبيع التجاري

جميع ما تعلمناه سابقاً من تعليمات (قراءة، كتابة، اسناد، شرط، تكرار) هي تعليمات كافية لحل أغلب المسائل التي تواجهنا وما سنتعلمه من الآن وصاعداً هي أدوات مساعدة تجعل كتابة الكود البرمجي أكثر سهولة.

المصفوفات

- تعتبر المصفوفات إحدى أهم بنى المعطيات الخطية، وذلك لأنها تقوم بتخزين مجموعة من العناصر بشكل خطي بحيث يمكن تعريف علاقة ترتيب بين هذه العناصر، أي أن هناك عنصر أول وثاني وثالث وهكذا..
- تكمن أهمية وفائدة المصفوفات عندما نحتاج إلى تعريف واستخدام عدد كبير من المتحولات من النوع نفسه، وهذا الأمر يتطلب جهد ووقت ويكون عرضة للأخطاء إذا قمنا بالتعامل معه بالطرق العادية كتعريف 100 متحول 100 مرة
- المصفوفة:** هي مجموعة من العناصر لها نفس النوع وعددها ثابت يتم تخزينها بشكل متتالي ضمن الذاكرة وجميع هذه العناصر لها نفس الحجم لأنها ذات نوع واحد ويوجد للمصفوفات نوعين:

1. المصفوفات الأحادية:

التصريح عن المصفوفة الأحادية:

نوع المصفوفة	اسم المصفوفة	عدد عناصر المصفوفة
Array-type	name	[number of elements]

1. اسم المصفوفة:

هو الاسم الذي سنتعامل معه في البرنامج حيث يمكننا تسمية المصفوفة بأي اسم لكن وفق قواعد تسمية المتغيرات التي تعرفنا عليها سابقاً

2. نوع المصفوفة:

هو النوع الذي سوف نعرف به المصفوفة فقد تكون رقمية أو حرفية فلو عرفنا مصفوفة من النوع integer فإن جميع عناصرها ستكون integer

3. عدد عناصر المصفوفة:

هو حجم المصفوفة الذي ستشغله في الذاكرة وقد يكون أي رقم حسب الاحتياج. ولكنه ثابت ولا نستطيع تغييره لاحقاً

كيف يمكننا الوصول إلى عنصر ضمن المصفوفة؟

يمكننا الوصول إلى أي عنصر ضمن المصفوفة بشكل مباشر دون الحاجة للمرور على العناصر السابقة له، بمجرد معرفة ترتيب العنصر ضمن عناصر المصفوفة (الأول أو الثاني..). ويسمى ترتيب العنصر ضمن المصفوفة بدليل المصفوفة (Index) ويبدأ هذا الدليل بالرقم (0) في أغلب لغات البرمجة مثل لغة C++ أي أن دليل العنصر الأول هو 0 ودليل العنصر الثاني هو 1 وهكذا

يمكننا إجراء العديد من العمليات على عناصر المصفوفة كالإدخال والإخراج والإسناد ونتعامل مع عناصرها على أنها متحولات عادية

تخزين المصفوفة:

لنكن لدينا مصفوفة مؤلفة من 100 عنصر معرفة وفق `int a[100]` حيث `a[0]` يعد أول عنصر و `a[99]` العنصر الأخير بحيث يتم ترتيب العناصر من 0 إلى 99 وليس من 1 إلى 100

وبما أن المصفوفة من النوع integer فإن كل متحول يحجز 4 bytes في الذاكرة

فإذا قمنا بتعريف مصفوفة `int a[100]` سيكون حجم هذه المصفوفة في الذاكرة يساوي $4 \times 100 = 400$ bytes

سؤال: ماذا يحدث إذا طلبنا من المستخدم إدخال العنصر `a[100]` من المصفوفة السابقة؟

في الواقع إن العنصر `a[100]` لا يعتبر من المصفوفة أساساً لأن آخر متحول في المصفوفة هو `a[99]` وفي البيئات المغيرة ل C++ لا يمكننا استخدامه ويعتبر ذلك runtime error، أما في C++ لدينا 3 حالات:

1. في حال كانت الذاكرة محجوزة عن طريق أحد البرامج المشغلة في الجهاز في مكان `a[100]` سيظهر لدينا runtime error لأننا نحاول الكتابة على برنامج الذاكرة فيه ممتلئة.
2. في حال كانت الذاكرة في ذلك المكان متاحة يمكننا استخدامها ولكن من الممكن أن يتم حجزها في وقت لاحق لبرنامج آخر وبالتالي فقدان البيانات المخزنة في تلك الخانة.
3. في حال كانت الذاكرة محجوزة من قبلنا في البرنامج نفسه فمثلاً لو عرفنا `x`, `int a[100]` وقمنا بالتعديل على العنصر `a[100]` من الممكن أن نكون قد عدلنا على قيمة `x` أيضاً في حال كانت تلك القيمة مخزنة في نفس خانة `a[100]` لذلك يجب علينا الانتباه من هذا الأمر ولا نتعدى حدود المصفوفة

أمثلة:

1- اكتب برنامج يقوم بإدخال مصفوفة وطباعها

```
1. #include <iostream>
2. using namespace std;
3. int main () {
4.     int a[100];
5.     for (int i = 0; i < 100; i++)
6.         cin >> a[i];
7.     for (int i = 0; i < 100; i++)
8.         cout << a[i] << endl;
9.     return 0;
10. }
```

نقوم بعملية الإدخال من خلال استخدام حلقة for بدلاً من إدخال كل عنصر على حدة (كما في السطر 5)

ونقوم بعملية الإخراج من خلال استخدام حلقة for بدلاً من إخراج كل عنصر على حدة (كما في السطر 7)

```
#include <iostream>
using namespace std;
int main () {
    int a[100], sum=0;
    for (int i = 0; i < 100; i++)
    {
        cin >> a[i];
        sum = sum + a[i];
    }
    cout << "Average: " << sum/100;
    return 0;
}
```



هل يمكن تعديل حجم المصفوفة أثناء عمل البرنامج (في الصيغة القياسية)؟

لا، لا يمكن بأي طريقة تعديل حجم أو نوع المصفوفة بعد التصريح عنها لا عن طريق المستخدم ولا حتى عن طريق المطور ولكن يمكن تغيير قيم العناصر الموجودة ضمنها، ويجب أن يكون حجمها إما معروف أو ثابت `const`

بما أن حجم المصفوفة ثابت لا يتغير فكيف يمكننا التعامل مع الحالات التي نحتاج إلى عدد مختلف من المصفوفة كل مرة؟

على سبيل المثال لدينا جامعة يدخل عليها سنوياً من 1000 إلى 2000 طالب كحد أقصى كيف نتعامل مع هذا الأمر؟

الجواب:

نستخدم الحد الأقصى اثناء تعريف المصفوفة ولو كان سيستهلك ذاكرة بشكل أكبر ولكن هنا سنضمن أن البرنامج سيعمل في جميع الحالات ويبقى على المستخدم أن يحدد العدد الأقصى من الطلاب عن طريق حلقة `for`

```
#include <iostream>
using namespace std;
int main () {
    int MaxNumber[2000];
    int CurrentNumber;
    cout << "Enter the number of students: ";
    cin >> CurrentNumber;
    cout << "Enter the ID of each student: " << endl;
    for (int i = 0; i < CurrentNumber; i++)
    {
        cin >> MaxNumber[i];
    }
}
```



في المثال السابق يمكن للمستخدم إدخال العدد الذي يريده دون التعديل على حجم المصفوفة

2. المصفوفات الثنائية:

هي سلسلة مصفوفات أحادية تمتلك نفس نوع data type
يمكن اعتبار المصفوفة متعددة الأبعاد على أنها مصفوفة مصفوفات، فمثلاً يمكن القول بأن المصفوفات الثنائية هي مصفوفة من المصفوفات الأحادية أو بعبارة أخرى جدول ثنائي البعد لبيانات لها نفس النوع
التصريح عن المصفوفات الثنائية:

نوع المصفوفة	اسم المصفوفة	[عدد الأعمدة] [عدد الأسطر]
Array-type	name	[rows] [columns]

ليكن لدينا `int a[4][3]` تمثل مصفوفة ثنائية مكونة من 4 أسطر و 3 أعمدة (4 مصفوفات تحوي كل منها على 3 عناصر)

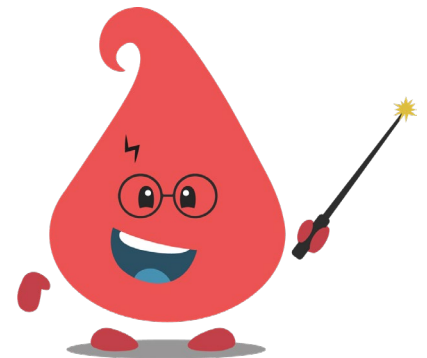
	0	1	2
0	[0][0]	[0][1]	[0][2]
1	[1][0]	[1][1]	[1][2]
2	[2][0]	[2][1]	[2][2]
3	[3][0]	[3][1]	[3][2]

لاحظ أننا كتبنا عدد أسطر المصفوفة أولاً ثم أعمدها، فإن قمنا بقلب أماكن الأسطر والأعمدة أثناء التصريح أي `int [3][4]` فإننا نحصل على المصفوفة المؤلفة من 3 أسطر و 4 أعمدة والتي تختلف عن المصفوفة السابقة
يمكن تعميم المصفوفات الثنائية لتصبح ثلاثية ورباعية مثل `int x[4][3][5]`; أربع مصفوفات ثنائية كل واحدة منهن تحوي ثلاث مصفوفات ثنائية في كل مصفوفة 5 عناصر

حقيقة: الذاكرة RAM لا تخزن المصفوفات الثنائية كما نتخيلها نحن على شكل أعمدة وأسطر وإنما تخزنها على شكل خانات متتالية

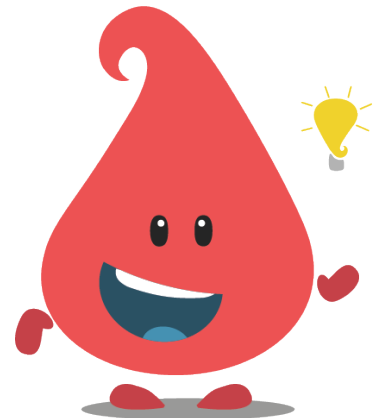
مثال: اكتب برنامج يقوم بإدخال عناصر مصفوفة أبعادها 4 و 3 ومن ثم طباعتها على الشاشة
في هذا المثال يمكننا كتابة

```
#include <iostream>
using namespace std;
int main () {
    int a[4][3];
    for (int j = 0; j < 3; j++)
    {
        cin >> a[0][j];
        cin >> a[1][j];
        cin >> a[2][j];
        cin >> a[3][j];
    }
}
```



ولكن الأسهل وضع الحلقة السابقة ضمن حلقة أخرى for لتكرارها

```
#include <iostream>
using namespace std;
int main () {
    int a[4][3];
    for (int i = 0; i < 4; i++)
    {
        for (int j = 0; j < 3; j++)
            cin >> a[i][j];
    }
    for (int i = 0; i < 4; i++)
    {
        for (int j = 0; j < 3; j++)
            cout << a[i][j] << endl;
    }
    return 0;
}
```



يمكن تشبيه المصفوفة الأحادية برف يحتوي على عدد معين من العلب
كما يمكن تشبيه المصفوفة الثنائية على أنها خزانة تحتوي رفوف عدة على كل رف عدد معين من العلب

ملاحظات:

نستخدم حلقة for لإدخال عناصر المصفوفة عنصر تلو الآخر فإذا كان لدينا مصفوفة `int x[100]` ونريد أن نطلب من المستخدم إدخال جميع عناصرها فلا يمكننا كتابة `cin>>x`

المترجم يحجز خانات متتالية من الذاكرة RAM عند تعريف مصفوفة أحادية كانت أم ثنائية ولا يقوم بحفظ سوى عنوان أول خانة من خانات المصفوفة وباقي الخانات يقوم بحسابها من فرق الخانات فمثلاً إذا كان لدينا `int x[10]` فإن المترجم لا يعرف سوى دليل (index) الخانة الأولى فقط `x[0]` وعندما نريد التعامل مع خانة `x[5]` (قراءة، كتابة، اسناد...) مثلاً فإن المترجم سيقوم بحساب الفرق بين `x[5]` و `x[0]` وسيعبر 5 خانات بعد `x[0]`. فيما يلي مثال عن اسناد قيمة 78 في `x[5]`

محتوى الخانة	200	7	3	14		78				
دليل الخانة	0	1	2	3	4	5	6	7	8	9

ولذلك لغة C++ لا تفرض فحص الحدود أثناء التنفيذ مما يسمح لنا بالكتابة على خانات خارج حدود المصفوفة وذلك لأن المترجم لا يعلم ما إذا كانت تلك الخانة ضمن حدود المصفوفة أو ليست ضمنها

وفي اللغات الأخرى (الأبطلأ من C++) لا يمكننا الكتابة على عنصر خارج المصفوفة لأن تلك اللغات تتميز بوجود آلية تحقق مما إذا كان العنصر المطلوب ضمن حدود المصفوفة أم لا، لكن هذا الأمر يستغرق وقتاً أطول مما يجعل البرنامج أبطأ